



← Back to student guides

 DVC

Data versioning

3 levels

113 sections



The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

CODE

POINTER

CACHE

REMOTE



Quick Start

 Open PDF

A one-page starter for this tool. Scroll through it here, or open it full screen.





Git-LFS

Solves the size problem, but still charges for bandwidth/storage per repo and has zero awareness of pipelines — it does



MLOps MENA Community

THE FIX, IN ONE LINE

DVC keeps the heavy data on remote storage like GCS, Azure — and leaves behind a tiny pointer in Git that holds only a hash.

Git for code. DVC for data. Same commands, same habits, one workflow.



MLOps MENA Community

THE PROBLEM → THE FIX

How Git and DVC Divide the Work



Part three of three. The problems at this level are rarely about syntax. They are about a remote sixty people depend on, a model nobody can rebuild, a storage bill nobody owns, and an erasure request against an immutable store. Start with the error table, then the practices, verification, and playbooks underneath it.

Common errors at this level

Cumulative. Everything from Beginner and Mid still applies. These cause incidents rather than failed runs.

Symptom	Real cause	Fix
<code>dvc gc --cloud</code> destroyed other branches' data	Default <code>--workspace</code> scope on a shared remote	Remove delete from human credentials; retention by lifecycle policy
Deleted remote objects are unrecoverable	Bucket versioning never enabled	Enable it. This is the highest-priority change
A corrupted object was restored silently	<code>verify</code> not set; a pointer is not proof	<code>dvc remote modify ... verify true</code>
An object at a hash path holds different bytes	Someone with write access replaced it	Restrict write; enable versioning; <code>verify true</code>
A leaked access key had data write access	Long-lived credential stored in CI	OIDC with a branch-pinned trust policy
A fork pull request overwrote a dataset	Fork build held a write credential	Read-only remote for forks; split the workflow
Cannot honour an erasure request	Personal data lives in the versioned store	Version derived or pseudonymised data; crypto-shred



Compliance-mode lock blocks a lawful deletion	Retention and erasure obligations collide	Architectural. Keep erasable data out of the immutable store
A model from last year cannot be rebuilt	The environment was never pinned	Hash-pinned lock file and image digest in <code>deps</code>
<code>dvc repro</code> succeeds but metrics differ	Undeclared randomness or unpinned wheels	Seed everything; declare the lock file
Storage bill tripled with no new datasets	Unreferenced sweep artifacts	Scheduled <code>exp remove</code> plus a lifecycle rule
Requests cost more than storage	A million small objects	Shard; measure request counts, not just bytes
Egress dominates the bill	Runners in a different region from the bucket	Co-locate compute and storage
A "cheap archive" is expensive to read	Cold class with high retrieval cost	Model retrieval, not just storage, before tiering
Nine forks of one canonical dataset	Upstream requests took weeks	Fix turnaround time; a registry is a product with an SLA
A consumer broke when upstream changed	Consumer tracked a moving path or branch	<code>dvc import --rev</code> a tag, <code>dvc update</code> deliberately
Self-hosted runner leaked between jobs	Persistent runner ran fork code	Ephemeral runners for pull requests
Nobody knows who owns a dataset	No CODEOWNERS, no README	Ownership recorded per dataset
"We are reproducible" is unverified	No scheduled rebuild	A weekly job that rebuilds a past release
A terabyte dataset is unusable through DVC	Wrong tool for the shape	Table format for tables; track the query and its hash



Versioning on, delete off

Bucket versioning enabled, and delete permission removed from every human credential. Together they make mistakes survivable.



OIDC everywhere

Nothing stored means nothing to leak, and the trust policy pins the repository and branch.



verify true on remotes

Re-hash on download. Corruption is refused instead of restored and reported as success.



Retention by lifecycle policy

Declarative, audited, and impossible to trigger from a stale clone. Never `gc --cloud` as a routine.



Pin the environment too

A hash-pinned lock file and an image digest in `deps`. Otherwise reproducibility stops at the language boundary.



Weekly rebuild of a past release

The only thing that turns "reproducible" from a claim into a monitored property.



Make cost visible

Bytes, objects, and unreferenced-experiment share, published where the team sees them.



Registry as a product

Dated immutable paths, tags per release, a README, an owner, and a turnaround promise.



01 **Recover an overwrite**

On a test bucket with versioning on, overwrite a DVC object by hand and restore the previous version. If you cannot do this, versioning is not enabled and that is your top priority.

02 **Watch `verify` refuse**

Corrupt one object in a test remote, run `dvc pull` without `verify` and note it succeeds with wrong data. Enable `verify true` and repeat.

03 **Read the destruction dry-run**

Run `dvc gc --cloud --dry-run --workspace` on a multi-branch repository and read the object count. Show that number to anyone holding delete permission.

04 **Prove the trust policy is scoped**

Try to assume your CI role from a throwaway branch and confirm the refusal. If it succeeds, the `sub` condition is not pinned.

05 **Rebuild something old**

Pick a release from three months ago, check it out on a clean machine, `dvc pull`, `dvc repro`, and compare the metrics. Note exactly where it broke.

06 **Find the unpinned layer**

List everything that revision does not pin: Python version, wheels, CUDA, image digest. Add the lock file and Dockerfile to `deps` and confirm a change invalidates the stage.

07 **Measure the real bill**

Total bytes, object count, and the fraction in experiment artifacts unread for three months. Calculate the lifecycle saving.

08 **Classify for erasure**



09

Run the checklist on someone else's project

The review checklist below, against a repository you did not write. It will find something.

The hardening pass every remote should get

BASH

Copy

```
# 1. Versioning: makes every other mistake survivable
aws s3api put-bucket-versioning --bucket ml-data-prod \
  --versioning-configuration Status=Enabled

# 2. Object lock in governance mode: deletion needs a specific privilege
aws s3api put-object-lock-configuration --bucket ml-data-prod \
  --object-lock-configuration '{"ObjectLockEnabled":"Enabled",
  "Rule":{"DefaultRetention":{"Mode":"GOVERNANCE","Days":365}}}'

# 3. Encryption at rest
aws s3api put-bucket-encryption --bucket ml-data-prod \
  --server-side-encryption-configuration \
  '{"Rules":[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"AES256"}}]}'

# 4. Access logging, so a mistake is traceable
aws s3api put-bucket-logging --bucket ml-data-prod --bucket-logging-status file://loggin

# 5. Lifecycle: retention as declarative policy, not a manual command
aws s3api put-bucket-lifecycle-configuration --bucket ml-data-prod \
  --lifecycle-configuration file://lifecycle.json
```

BASH

Copy

```
# .dvc/config - committed, no credentials
[core]
  remote = origin
  autostage = true
```



```
jobs = 32
verify = true                # corruption refused, not restored
sse = AES256
['remote "archive"']
  url = s3://ml-data-archive/dvc
  verify = true
[cache]
  type = reflink,copy
  protected = true           # in-place edits fail loudly
```

Control	Prevents
Bucket versioning	An unrecoverable overwrite or delete
Object lock (governance)	Casual deletion; a <code>gc --cloud</code> accident
<code>verify true</code>	A corrupted or replaced object being restored silently
<code>cache.protected true</code>	An in-place edit rewriting cached history
Encryption at rest	A compliance finding
Access logging	An untraceable incident
Lifecycle policy	Unbounded storage growth, and manual <code>gc</code>

Three permissions to audit today

Who can **delete** on the remote, ideally nobody human. Who can **write** to the production prefix, ideally only CI on a protected branch. What identity a **fork pull request** receives, ideally read-only or nothing. Those three answers determine your entire blast radius.

Verifying, not assuming



BASH

Copy

```
# Is versioning actually on?
aws s3api get-bucket-versioning --bucket ml-data-prod

# Does verify catch a corrupted object?
dvc pull --verify

# Can a developer credential delete?
aws s3api delete-object --bucket ml-data-prod --key dvc/files/md5/xx/test 2>&1 | tail -1

# Is anything sensitive in the committed config?
git log -p --all -- .dvc/config | grep -iE 'key|secret|token|password' || echo clean

# Is the environment genuinely pinned?
grep -c 'sha256:' requirements.lock
dvc dag --outs | grep -q requirements.lock && echo "declared as a dep"

# Would an old release rebuild?
git checkout <old-tag> && dvc pull && dvc repro --dry

# How much would a cloud gc destroy?
dvc gc --cloud --dry-run --workspace | wc -l
```

Test that the control refuses something

After enabling object lock, try to delete an object without the bypass privilege. After setting `verify true`, corrupt an object and confirm the pull fails. After pinning the trust policy, try to assume the role from another branch. A control you have never watched refuse anything is decoration.

Automate the mechanical checks so they do not depend on anyone remembering:

.github/workflows/dvc-checks.yml

Copy

```
- name: dvc.lock must not be stale
  run: test "$(dvc status --json)" = "{}"

- name: dvc.lock must accompany dvc.yaml changes
  run: |
```



```
- name: No credentials in the committed config
run: |
  grep -iE 'access_key|secret|password|token' .dvc/config \
    && { echo "::error::credential in .dvc/config"; exit 1; } || true

- name: Bucket versioning must be enabled
run: |
  test "$(aws s3api get-bucket-versioning --bucket ml-data-prod --query Status --output
```

Cost governance

BASH

Copy

```
# Where the bytes are
aws s3 ls --summarize --human-readable --recursive s3://ml-data-prod/dvc/ | tail -3

# DVC-side hygiene
du -sh .dvc/cache
find .dvc/cache -type f | wc -l
dvc exp remove --all
dvc gc --dry-run --all-commits
```

Lever	Effect	Note
<code>push: false</code> on regenerable outputs	Often the largest single saving	Costs nothing but a config line
Scheduled experiment cleanup	Removes the biggest silent growth	<code>exp remove</code> then <code>gc</code>
Lifecycle to a colder class	Cheap long-term retention	Model retrieval cost too



Shard small files	Cuts request charges and hash time	Also speeds the data loader
Co-locate compute and storage	Removes egress	Often exceeds the storage line
Tier per output with <code>remote:</code>	Hot and cold in one pipeline	No manual moving

Publish two numbers and behaviour changes

Total bytes per project, and the fraction of those bytes in experiments nobody has read in ninety days. That second number is usually a majority, and it is the most persuasive thing you can put in front of a team, far more effective than a policy document about cleaning up.

Incident playbooks

`dvc gc --cloud` deleted live data

Stop all pushes immediately, because a subsequent push can overwrite the delete markers you need to recover. Restore from bucket versioning. This is the entire reason versioning is non-optional. Then remove delete permission from every human credential and move retention to a lifecycle policy. The post-mortem action is a permission change, not a training session.

A remote object is corrupt or missing

Confirm it with `dvc pull -v`, which names the hash and the remote. Check bucket versioning for a previous version of that key; if present, restore and you are done. If not, find any machine or CI cache that still holds the object and push from there. If nothing has it, identify every commit referencing the hash, rebuild the artifact if its inputs survive, and record a



A model cannot be reproduced

Work the layers in order, cheapest first: is the commit findable from the model's hash, does `dvc pull` succeed for every dep in `dvc.lock`, does `dvc repro --dry` report up to date, and does an actual rerun produce matching metrics. The break is almost always the last two, and almost always the environment. Fix it by adding the missing pin to `deps`, not by documenting the discrepancy.

The cache filled the disk mid-sweep



0m Stop the queue

`dvc queue stop`. Do not delete yet, because a half-written cache object is worse than a full disk.



2m Measure before deleting

`du -sh .dvc/cache` and `dvc gc --dry-run --all-commits`. Know what is reachable.



5m Drop experiment refs first

`dvc exp remove --all` makes sweep artifacts collectable, then `dvc gc --all-commits`.



10m Move the cache

`dvc cache dir --local /mnt/big/cache` rather than fighting a small root disk.



after Close the loop

Mark regenerable intermediates `push: false`, budget disk per sweep, schedule cleanup.

A credential leaked

Rotate before anything else. It may already be in a log, a cache, or a fork. Then audit the bucket's access logs for the exposure window, looking for **writes and deletes** rather than only reads, because a write is the damaging case. Replace the mechanism, not just the value: if a long-lived key leaked, the fix is OIDC. Rotating a key you still store only resets the clock.



Establish the blast radius: which hashes contain the data, and which commits, tags, and models reference those hashes. Decide what must survive, usually the model and its metrics remain auditable while the training data goes, and write that decision down. Rebuild the dataset without the subject, push it, and update pointers on active branches. Delete the old objects from the remote, every local cache, every CI cache, and any mirror, including old versions in a versioned bucket. Record a committed tombstone naming the erased hash, the date, and the authority. Then close the class by moving personal data out of the versioned store.

Running DVC as a platform

Publish a template repository, not documentation. A working pipeline, the CI workflow with OIDC and the `dvc status` gate, and a `.dvc/config` with the right remote, `verify`, `jobs`, and cache settings. Teams inherit every good decision by cloning; a wiki page is read once.

Own the remote layout and its policy. One prefix per environment, versioning on, delete removed from humans, lifecycle rules for retention, access logging. Teams should not each invent this.

Make the safe path the fast path. If `dvc pull` from the sanctioned remote is fast and documented while copying from a colleague's laptop is slow, nobody copies from laptops. Regional co-location and a tuned `jobs` do more for compliance than a policy.

Measure and publish. Without numbers, "our data is under control" is an opinion:

Metric	Why
Bytes and objects per project, with trend	Cost and growth
Share of bytes in unreferenced experiments	The easiest saving
Percentage of pipelines with a clean <code>dvc status</code> in CI	Whether reproducibility is real



Age of the oldest successfully reproduced release	The genuine reproducibility horizon
Remotes with human write access	The blast radius
Median <code>dvc pull</code> time in CI	Developer and CI friction

The failure mode of a shared registry

A team needs one extra column, cannot get it quickly, and forks. Six months later there are nine variants and no canonical source. The fix is **turnaround time on requests**, not policy. If a reasonable schema change takes three weeks, forking is rational and you will lose. Treat it as a product with a service level, or do not call it canonical.

Machine-learning specifics

Weights are data, not code. Track them, but keep them out of the same lifecycle as source. A retrain should not require a code release, and a code fix should not require re-pushing gigabytes.

Model artifacts accumulate fastest. Every experiment caches one. A 2 GB model across thirty sweeps a month is where the storage bill comes from, and `dvc exp remove` plus a lifecycle rule is the answer.

Register promoted models explicitly. A Git tag plus `dvc.lock` gives you data, code, and metrics for a release. `live.log_artifact(..., type="model")` makes the artifact discoverable rather than implied.

Log the environment alongside the metrics. Git SHA, image digest, and the input data hash written into the run's parameters. Without those three, a reproducible pipeline is still an unreproducible model.

Seed everything and say so. Python, NumPy, the framework, any shuffling, and the split. An unseeded run is an unreproducible run no matter how well the data is versioned.



```
live.log_params({
  "git_sha": git_sha(),
  "image_digest": os.environ.get("IMAGE_DIGEST", "unknown"),
  "data_hash": lock_hash("data/train.parquet"),
  "seed": SEED,
})
```

The checklist to run before shipping

Check	Looking for
<code>dvc.lock</code> committed, never hand-edited	The record of what ran
Every script, import, and lock file in <code>deps</code>	No silent skips, environment pinned
Metrics and plots <code>cache: false</code>	Diffable from Git with no pull
No credentials in <code>.dvc/config</code> , ever in history	<code>git log -p</code> comes back clean
Regenerable intermediates <code>push: false</code>	Bounded storage cost
External deps are dated, immutable paths	A source cannot rewrite your history
<code>verify true</code> on remotes	Corruption refused, not restored
Bucket versioning enabled	Every mistake is survivable
No human credential can delete	The <code>gc --cloud</code> blast radius
CI uses OIDC with a branch-pinned trust policy	Nothing to leak, nothing to borrow
Fork pull requests are read-only	A contributor cannot overwrite data
<code>dvc status</code> gate in CI	The lock cannot drift from the code



Retention is a lifecycle policy	Audited, not clone-triggered
Small-file datasets sharded	Hashing and request costs bounded
No erasable personal data in versioned artifacts	The hard problem, before it is urgent
A scheduled reproduction check exists	Reproducibility monitored, not assumed
Each dataset has an owner and a README	Accountability and usability
Storage cost is visible to the team	Growth gets managed
The right tool for each data shape	Tables in a table format, not in DVC

Most data incidents are prevented at review time rather than at recovery time. Reading a DVC project for what it **guarantees**, rather than what it does, is the highest-leverage habit you can build.



PREVIOUS

Senior · Interview Review





LEARN

[Roadmaps](#)

[Student Guides](#)

[Courses](#)

[The MLOps Practitioner](#)

[Sessions](#)

[Articles](#)

COMMUNITY

[Meet the team](#)

[Services for companies](#)

[FAQ](#)

[Contact us](#)

[Privacy Policy](#)

WHERE WE ARE

[WhatsApp](#)

[Discord](#)

[LinkedIn](#)

[X](#)

[YouTube](#)

[GitHub](#)

[Zomra](#)



Brainsmingle space

Free to join this week with code **FREECORETEAM**



MLOps MENA
COMMUNITY

العربية



Join the Community ↗

